Ingegneria del Software — Soluzione del Tema 28/08/2023

Esercizio 1

Si consideri la seguente classe `CityMap` che memorizza i punti di interesse di una città (`Site`), organizzati per tipo (ristoranti, hotel, musei, etc.).

```
public class CityMap {
   // Ritorna l'insieme di tutti i punti di interesse memorizzati
   public /*@ pure @*/ Set<Site> getSites() { ... }

   // Aggiunge il punto di interesse passato per parametro
   public void addSite(Site site) { ... }

   // Ritorna tutti i punti di interesse del tipo passato per parametro,
   // ordinati per distanza crescente rispetto alla location loc indicata.
   // Lancia una NotFoundException se non esiste alcun punto di interesse del tipo indicato.
   public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }

   // Ritorna i k punti di interesse piu' vicini alla location loc indicata
   // (per semplicita', si assuma che non possano esistere due punti equidistanti da loc).
   // Se i punti di interesse sono meno di k, vengono ritornati tutti.
   // La mappa ritornata contiene i punti di interesse come chiave e la loro distanza da loc come valore.
   public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
}

public /*@ pure @*/ class Site {
   // Ritorna la posizione geografica del punto di interesse
   public Location getLocation() { ... }

   // Ritorna il tipo del punto di interesse (ristorante, hotel, museo, ...)
   public Type getType() { ...}
... }

public /*@ pure @*/ class Location {
   // Calcola la distanza rispetto alla Location passata per parametro
   public double getDistance(Location other) { ... }
... }
```

Domanda a)

Si specifichi in JML il metodo `getSitesByType`.

Soluzione

```
//@ requires type!=null && loc!=null
//@
//@ ensures (\exists Site s; getSites().contains(s); s.getType().equals(type)) && \result!=null &&
//@ (\forall Site s; ; \result.contains(s) <==> getSites().contains(s) && s.getType().equals(type)) &&
//@ (\forall int i; i>=0 && i<\result.size();
//@    (\forall int j; j>=0 && j<i; !\result.get(j).equals(\result.get(i)) &&
//@       \result.get(j).getLocation().getDistance(loc)<=\result.get(i).getLocation().getDistance(loc) ) )
//@
//@ signals (NotFoundException e) !(\exists Site s; getSites().contains(s); s.getType().equals(type))
public /*@ pure @*/ List<Site> getSitesByType(Type type, Location loc) throws NotFoundException { ... }
```

Domanda b)

Si specifichi in JML il metodo `getNearestSites`.

Soluzione

```
//@ requires k>0 && loc!=null
//@
//@ ensures \result!=null && \result.size() == (k<getSites().size() ? k : getSites().size()) &&
//@ (\forall Site s; \result.keySet().contains(s);
//@    getSites().contains(s) && \result.get(s)==s.getLocation().getDistance(loc) ) &&
//@ (\forall Site s; getSites().contains(s);
//@    (\num Site s1; getSites().contains(s1) && !s1.equals(s);
//@       s1.getLocation().getDistance(loc) < s.getLocation().getDistance(loc) ) < k
//@    ==> \tesult.keySet().contains(s) )
public /*@ pure @*/ Map<Site, Double> getNearestSites(int k, Location loc) { ... }
```

Domanda c)

Si consideri un'implementazione che fa uso di una mappa per memorizzare i punti di interesse organizzati per tipo, come mostrato di seguito.

```
public class CityMap {
   private Map<Type, Set<Site>> sites;
... }
```

Per tale implementazione di forniscano l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

L'invariante di rappresentazione (RI) deve specificare che la mappa non sia nulla, non contenga chiavi nulle, non contenga valori nulli o vuoti. Inoltre ogni punto di interesse deve essere non nullo e memorizzato nell'insieme corrispondente al proprio tipo.

RI:

```
//@ private invariant sites!=null && !sites.containsKey(null) &&
//@ (\forall Type t; sites.containsKey(t); sites.get(t)!=null && !sites.get(t).isEmpty() &&
//@    (\forall Site s; sites.get(t).contains(s); s!=null && s.getType().equals(t)) )
```

La funzione di astrazione (AF) si limita a definire `getSites` in funzione di `sites`, in quanto tutti gli altri metodi sono specificati a partire da `getSites`.

AF:

```
//@ private invariant (\forall Site s; ; getSites().contains(s) <==>
//@    sites.containsKey(s.getType()) && sites.get(s.getType()).contains(s) )
```

Domanda d)

Si consideri una classe `MergeableCityMap`, che aggiunge un metodo `void merge(CityMap other)` che aggiunge tutti i punti di interesse presenti in `other`. Può `MergeableCityMap` essere definita come sottoclasse di `CityMap` in accordo con il principio di sostituzione di Liskov? Si motivi la risposta.

Soluzione

`MergeableCityMap` può essere definita come sottoclasse di `CityMap` in quanto aggiunge un nuovo metodo senza modificare metodi presenti. Il nuovo metodo non viola alcuna proprietà della classe `CityMap`: infatti, invocare il metodo `merge` è equivalente ad aggiungere i punti di interesse singolarmente mediante molteplici invocazioni del metodo `addSite`.

Esercizio 2

Si consideri la seguente classe `SensorReadings` per memorizzare le letture di un sensore ambientale:

```
public class SensorReadings {
  private int id;
  private List<Double> data;
  private int daysInUse;
  public SensorReadings(int id) {
    this.id = id;
    data = new ArrayList<Double>();
    daysInUse = 0;
  }
  public void addReading(double d) {
    synchronized(data) { data.add(d); }
  }
  public double getReading(int index) {
    synchronized(data) { return data.get(index); }
  }
  public void changeAllReadings(List<Double> newData) {
    synchronized(data) { data = newData; }
  }
  public synchronized void newDayStarted() {
    daysInUse++;
  }
}
```

Domanda a)

La classe è correttamente sincronizzata? In caso negativo si spieghi quali siano i problemi e si proponga una soluzione a tali problemi che massimizzi il parallelismo tra i metodi.

Soluzione

L'accesso all'attributo privato `id` non va sincronizzato (l'attributo non cambia mai valore). Problematica è invece la sincronizzazione del metodo `changeAllReadings` che cambia il riferimento all'oggetto `data` su cui avviene la sincronizzazione del metodo stesso e dei metodi `addReading` e `getReading`. Volendo, come richiesto, mantenere la possibilità di accesso parallelo a tali metodi rispetto al metodo `newDayStarted`, si può risolvere introducendo un nuovo attributo `dataLock`, riferimento ad un oggetto che non cambia mai, per poi usare quello come punto di sincronizzazione dei tre metodi: `changeAllReadings`, `addReading` e `getReading`, come proposto nella seguente versione della classe:

```
public class SensorReadings {
  private int id;
  private List<Double> data;
  private Object dataLock;
  private int daysInUse;
  public SensorReadings(int id) {
    this.id = id;
    data = new ArrayList<Double>();
    dataLock = new Object();
    daysInUse = 0;
  }
  public void addReading(double d) {
    synchronized(dataLock) { data.add(d); }
  }
  public double getReading(int index) {
    synchronized(dataLock) { return data.get(index); }
  }
  public void changeAllReadings(List<Double> newData) {
    synchronized(dataLock) { data = newData; }
  }
  public synchronized void newDayStarted() {
    daysInUse++;
  }
}
```

Domanda b)

Si aggiunga alla classe `SensorReadings` un nuovo metodo pubblico: `void waitfor30()` che sospende il chiamante se ci sono meno di 30 letture nella lista `data`.

Soluzione

Alla luce delle modifiche proposte nella soluzione alla precedente domanda, il metodo deve essere così formulato:

```
public void waitfor30() throws InterruptedException {
  synchronized(dataLock) {
    while(data.size()<30) dataLock.wait();
  }
}
```

Bisogna inoltre aggiungere la chiamata: `dataLock.notifyAll()` alla fine dei metodi `changeAllReadings` e `addReading`.

Domanda c)

Si aggiunga un nuovo metodo pubblico: `void readyToBeReplaced()` che sospende il chiamante fino a quando non ci siano almeno 10000 letture nella lista `data` oppure siano passati meno di 365 giorni di utilizzo del sensore. Si chiarisca se e come deve cambiare la sincronizzazione degli altri metodi.

Soluzione

La condizione di sospensione riguarda tanto l'attributo `data` quanto l'attributo `daysInUse`, il cui accesso è stato fin qui protetto usando due "lock" diversi (`dataLock` e `this`). Per come funziona la sincronizzazione di Java questo approccio non si può più seguire ed è quindi necessario usare un unico "lock" per tutto. Ad esempio, scegliendo `this`, si può giungere alla seguente soluzione:

```
class SensorReadings {
  private int id;
  private List<Double> data;
  private int daysInUse;
  public SensorReadings(int id) {
    this.id = id;
    data = new ArrayList<Double>();
    daysInUse = 0;
  }
  public synchronized void addReading(double d) {
    data.add(d); notifyAll();
  }
  public synchronized double getReading(int index) {
    return data.get(index);
  }
  public synchronized void changeAllReadings(List<Double> newData) {
    data = newData; notifyAll();
  }
  public synchronized void newDayStarted() {
    daysInUse++; notifyAll();
  }
  public synchronized void waitfor30() throws InterruptedException {
    while(data.size()<30) wait();
  }
  public synchronized void readyToBeReplaced() throws InterruptedException {
    while(data.size()<10000 || daysInUse<365) wait();
  }
}
```

Esercizio 3

Si consideri il seguente metodo:

```
1  public static void test(int x,int y) {
2    if (x<=0 || y<=0) return;
3    y=y+x;
4    while (y>x && y/x>x) {
5      if(y%2==0)
6        x++;
7      y++;
8    }
9  }
```

Domanda a)

Si determini un insieme minimo di casi di test per garantire la copertura delle istruzioni (statement coverage).

Soluzione

(0,0) per coprire il return, (1,1) per tutto il resto.

Domanda b)

Si determini un insieme minimo di test per garantire la copertura delle decisioni (edge coverage).

Soluzione

I due test precedenti non coprono il ramo false dell'if più interno. Fermo restando la necessità di mantenere il primo dei due casi precedenti per coprire il ramo true del primo if, si può formulare un unico caso di test che, sfruttando il ciclo, copra i due rami dell'if interno al ciclo, come ad esempio il test (1,2).

Domanda c)

Si determini un insieme minimo di test per garantire la copertura delle decisioni e delle condizioni (edge and condition coverage).

Soluzione

Usando i casi precedenti manca ancora la copertura di due condizioni: Per la prima, nell'if più esterno, basta (1,0); per la seconda, si dovrebbe coprire il caso false di $y > x$, ma questo è impossibile in quanto $y > x$ all'entrata nel ciclo e durante l'esecuzione y aumenta sempre almeno quanto x.

Esercizio 4

Si considerino le seguenti dichiarazioni di classi Java.

```
abstract class Vehicle {
  abstract void start();
}

class Car extends Vehicle {
  void start() {
    System.out.println("Car started");
  }
  void start(boolean remoteControl) {
    if (remoteControl) {
      System.out.print(" Car started with remote");
    } else { System.out.print("missing remote, so ");
      start();
    }
  }
}

class HybridCar extends Car {
  void start() {
    System.out.println("Hybrid started");
  }
}

class ElectricCar extends HybridCar {
  void start(boolean remoteControl) {
    if (!remoteControl) {
      System.out.println("starting normally: ");
      super.start();
    }
    super.start(remoteControl);
  }
  void start() {
    System.out.println("Electric started");
  }
}
```

```
1  class Main {
2    public static void main(String[] args) {
3      Vehicle car = new Car();
4      HybridCar hybridCar = new HybridCar();
5      Car electricCar = new ElectricCar();
6      car.start();
7      car.start(true);
8      car.start(false);
9      hybridCar.start();
10     hybridCar.start(true);
11     hybridCar.start(false);
12     electricCar.start();
13     electricCar.start(true);
14     electricCar.start(false);
15   }
16 }
```

Domanda a)

Ci sono linee del programma non valide? In caso affermativo indicarle, motivando brevemente la risposta e precisando se causano errori a tempo di compilazione o di esecuzione.

Soluzione

Le righe 7 e 8 non sono valide, in quanto Car ha tipo statico Vehicle, che non ha il metodo start(boolean)

Domanda b)

Eliminate le righe che generano gli errori individuati al punto precedente, indicare cosa stampa il programma, motivando brevemente la risposta. Riportare sempre il numero di riga riportato nel testo.

Soluzione

6 Car started
9 Hybrid started
10 Car started with remote
11 Missing remote, so Hybrid started
12 Electric started
13 Car started with remote
14 Starting normally: Hybrid started
Missing remote, so Electric started